

The Prompt Ladder

FL-03 — General AI Fluency, Week 2

Abdul Salam

Backend & AI Systems Engineer

Baseline — The Weak Prompt

"Write backend code"

Output (representative excerpt):

```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/api/data', methods=['GET'])
def get_data():
    return jsonify({"message": "Hello, World!"})

if __name__ == '__main__':
    app.run(debug=True)
```

Followed by a paragraph explaining what Flask is and offering to add more features “if needed.”

Genuinely useless: wrong framework (Flask, not the FastAPI stack actually in use), unrelated to any real task, and it hedges with a generic explainer instead of doing something specific.

Version 1 — added: a defined audience + a clearer goal

"Write backend code for a REST API endpoint that lets a user create a new task, for a junior developer learning FastAPI."

Output (representative excerpt):

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Task(BaseModel):
    title: str
    completed: bool = False

tasks = []

@app.post("/tasks")
def create_task(task: Task):
    tasks.append(task)
    return {"message": "Task created", "task": task}
```

What changed in the prompt:

Added “for a junior developer learning FastAPI” (audience) and named the actual action, “create a new task” (goal).

What improved in the output:

Correct framework this time, an actual working endpoint instead of a generic hello-world.

What still failed:

Invented its own schema (“completed” instead of “done”), no status code specified, wraps the response oddly instead of following REST convention.

What to try next:

Give it real context about the existing project instead of letting it invent a fresh schema.

Version 2 — added: real context

"...for a junior developer learning FastAPI. This is for a to-do list app already using an in-memory list of tasks with fields id, title, done."

Output (representative excerpt):

```
tasks = [{"id": 1, "title": "Buy milk", "done": False}]
next_id = 2

@app.post("/tasks")
def create_task(task: dict):
    global next_id
    new_task = {"id": next_id, "title": task["title"], "done": False}
    tasks.append(new_task)
    next_id += 1
    return new_task
```

What changed in the prompt:

Gave it the actual existing schema (id, title, done) instead of letting it guess.

What improved in the output:

Field names now match the real project instead of inventing “completed.”

What still failed:

No Pydantic validation (uses a raw dict), no status code, no handling for a missing or empty title.

What to try next:

Specify the exact output format and constraints instead of leaving structure open.

Version 3 — added: specified output format + constraints

"...Return only the FastAPI route function using Pydantic for validation, under 15 lines, matching REST convention (POST /tasks, 201 status code)."

Output (representative excerpt):

```
class TaskCreate(BaseModel):
    title: str

@app.post("/tasks", status_code=201)
def create_task(task: TaskCreate):
    global next_id
    new_task = {"id": next_id, "title": task.title, "done": False}
    tasks.append(new_task)
    next_id += 1
    return new_task
```

What changed in the prompt:

Explicitly required Pydantic, a line limit, and the correct status code.

What improved in the output:

Proper validation model, correct 201, clean and short — genuinely close to production-usable.

What still failed:

No input validation for an empty or missing title (would need a 400), no note on how to test it.

What to try next:

Give it a real example of my own style so naming and structure match my actual codebase, not just “correct FastAPI.”

Version 4 — added: an example of what good looks like

"...Here's an example of my style: @app.get('/tasks/{task_id}') with a 404 raised via HTTPException if not found. Match this pattern."

Output (representative excerpt):

```
from fastapi import HTTPException

@app.post("/tasks", status_code=201)
def create_task(task: TaskCreate):
    global next_id
    if not task.title.strip():
        raise HTTPException(status_code=400, detail="Title is required")
    new_task = {"id": next_id, "title": task.title, "done": False}
    tasks.append(new_task)
    next_id += 1
    return new_task
```

What changed in the prompt:

Gave it one real snippet showing my HTTPException pattern instead of describing style abstractly.

What improved in the output:

It inferred the validation approach on its own and matched it correctly — the missing 400 from Version 3 just appeared, unprompted.

What still failed:

Still no note on how to actually verify this works.

What to try next:

Ask explicitly for verification steps and one edge case to test.

Version 5 — added: quality criteria + verification requirements

"...Include the exact curl command to test it, and name one edge case it should reject."

Output (representative excerpt):

```
# Edge case handled: empty title returns 400, not a silently-created blank task.  
  
# Test:  
# curl -X POST http://localhost:8000/tasks -H "Content-Type: application/json" -d '{"title":""}'  
# Expected: 400 {"detail":"Title is required"}
```

What changed in the prompt:

Asked for verification steps and one named edge case, not just code.

What improved in the output:

Now includes exactly how to prove it works and what failure mode it guards against — genuinely reusable by someone else, not just by me.

What still failed:

Honestly, nothing major left for this scope — this is close to done.

What to try next:

This prompt structure is now reusable as a template for any endpoint, not just this one.

Final Reusable Prompt

Write backend code for [specific endpoint/action], for [audience]. Context: [real existing schema/stack]. Return only [format], under [X] lines, matching [convention]. Match this style: [real snippet from my codebase]. Include the exact test command and name one edge case it should reject.

The Honest Moment

Version 1 to Version 2 was the biggest jump. Adding real context stopped the model from inventing a fake schema, and that's the one step that would have quietly broken every later version if it had been skipped, since each subsequent version kept building on whatever field names came out of that step.